

求解代码详细拆解

四个问题的建模代码：模块职责、数据结构、算法实现与调用链 ——逐文件、逐函数的技术说明

2026 高教社杯全国大学生数学建模竞赛 B 题

2026 年 9 月 11 日

摘要

本文档逐文件、逐函数拆解本题的全部求解代码。核心求解代码共 6 个模块、约 2700 行：几何内核 `geom_core.py` (384 行)、问题二优化 `q2_solver.py` (316 行)、本地模拟器 `simulator.py` (416 行)、策略内核 `robot_core.py` (1087 行)、实验数据生成 `make_data.py` (347 行)、正式测试执行器 `run_http_tests.py` (103 行)；另有绘图 `make_figures.py` (1003 行) 与字号审计 `audit_figures.py` (60 行)。

文档给出：模块依赖关系、四个核心数据结构的精确契约、每个公开函数的输入输出与算法复杂度、一次完整动作从决策到落盘的调用链、以及让代码可移植的工程约束（源码纯 ASCII、中文标签外置）。

目录

1 总览	2
1.1 模块地图	2
1.2 依赖关系	2
2 四个核心数据结构的契约	3
2.1 半平面三元组 (a, b, c)	3
2.2 观测四元组 <code>obs[c]</code> 中的元素	3
2.3 估计三元组 <code>est[c] = (x, y, sigma)</code>	3
2.4 任务四元组 (kind, channel, position, extra)	3
3 <code>geom_core.py</code> 逐函数拆解	3
3.1 基础工具 (40–62 行)	3
3.2 定位区域核心 (68–260 行)	4
3.3 直径与圆覆盖 (265–347 行)	4
3.4 最小包围圆 (349–403 行)	4
3.5 问题二辅助 (404–448 行)	5
4 <code>q2_solver.py</code> 拆解	5

4.1	函数清单	5
4.2	求解流程	5
4.3	基线策略的定义	5
5	robot_core.py 逐层拆解	5
5.1	第 1 层：两个客户端（55–140 行）	6
5.2	第 2 层：认证判据的原语（142–167 行）	6
5.3	第 3 层：参数与状态（169–250 行）	6
5.4	第 4 层：站位生成（269–336 行）	7
5.5	第 5 层：认证扫描（337–377 行）	7
5.6	第 6 层：估计与信息获取（413–592 行）	7
5.7	第 7 层：末段清除（593–733 行）	8
5.8	第 8 层：任务规划与主循环（765–1145 行）	8
6	一次完整动作的调用链	9
7	复杂度与性能	9
8	工程约束与规范	9

1 总览

1.1 模块地图

模块	行数	职责
geom_core.py	384	纯几何内核：角楔 → 半平面、定位区域顶点枚举、有界性判据、直径（暴力 + 卡壳）、Thales 圆覆盖判据、最小包围圆（Welzl）
q2_solver.py	316	问题二：可行源集合采样、最坏情形直径、min-imax 求解、基线对照、Monte-Carlo
simulator.py	416	本地模拟器：物理解算（Arena）+ 附件 2 的 HTTP 协议层 + 算例生成
robot_core.py	1087	策略内核：两个客户端、信息状态与估计、信息获取、末段清除、任务规划、主循环
run_http_tests.py	103	正式测试执行器：起服务、走 HTTP、导出日志与统计
make_data.py	347	生成问题一 ~ 四的全部数值结果（CSV/JSON）与灵敏度实验
make_figures.py	1003	22 张插图的绘制（含外置图例辅助函数）
audit_figures.py	60	插图等效字号审计（解析 \includegraphics 宽度 + PNG 像素宽）

1.2 依赖关系

```

1 geom_core <-- q2_solver <-- make_data / make_figures
2   ^           ^
3   |           |
4 robot_core ----+          simulator <-- robot_core (LocalClient)
5   ^           ^
6   |           |
7 run_http_tests -----+ (HTTP) -----+
8                       |
9 make_data / make_figures -----+
10
11 import 关系（自上而下）：
12 simulator          -> （无内部依赖）
13 geom_core          -> （无内部依赖）
14 q2_solver           -> geom_core
15 robot_core          -> geom_core, simulator(仅常量)
16 run_http_tests      -> simulator, robot_core
17 make_data           -> geom_core, q2_solver, simulator, robot_core
18 make_figures        -> geom_core, q2_solver, simulator, robot_core

```

代码 1 模块依赖（箭头表示 import）

设计上刻意让 `geom_core` 与 `simulator` 都不依赖任何其他内部模块，这样几何算法可以脱离模拟器单独单元测试（`q1_smoke.py`、`q1_scan.py` 就是这么做的）。

2 四个核心数据结构的契约

代码里几乎所有跨函数传参都围绕下面四种数据，先把它们的”形状”钉死，后面看函数就很快。

2.1 半平面三元组 (a, b, c)

表示闭半平面 $\{(x, y) : ax + by \leq c\}$ 。角楔”S 处示向度 θ 、容差 ε ”由两条边界直线交叉成两个半平面：

```
1 t1, t2 = radians(theta - eps), radians(theta + eps)
2 a1, b1 = sin(t1), -cos(t1)      # 由 cross(u(theta-eps), v) >= 0 推出
3 hp1 = (a1, b1, a1*sx + b1*sy)
4 a2, b2 = -sin(t2), cos(t2)     # 由 cross(u(theta+eps), v) <= 0 推出
5 hp2 = (a2, b2, a2*sx + b2*sy)
6 return [hp1, hp2]
```

代码 2 角楔 $\rightarrow$ 半平面 (geom_core.wedge_halfplanes)

全部约束统一成” $\leq$ ”方向的好处：求交只需枚举直线交点再逐点检验（不需要处理 min/max 混合的半平面），实现短且不易错。

2.2 观测四元组 obs[c] 中的元素

每个频道的观测列表 self.obs[c] 存 (x, y, bearing_deg, t)：检测点坐标、该点的示向度读数、虚拟时刻。归航时取最后一个元素的方位角作为前进方向。

2.3 估计三元组 est[c] = (x, y, sigma)

由观测列表算出：(x, y) 是定位区域最小包围圆的圆心，sigma 是它的半径。当观测不足或区域退化时，退化为”沿最后一条射线、距离取 0.6 倍可行射线长度”的估计。

2.4 任务四元组 (kind, channel, position, extra)

kind $\in \{\text{"clear", "localise", "stop", "probe"}\}$ ；channel 为 None 表示不针对特定频道（如 stop）；position 是执行该任务的位置；extra 存放任务专属参数（例如 localise 的偏转角）。

3 geom_core.py 逐函数拆解

3.1 基础工具（40–62 行）

函数	说明
norm360(a)	角度归一化到 [0, 360)
deg2rad(a)	角度 $\rightarrow$ 弧度
bearing(px, py, qx, qy)	从 p 指向 q 的方位角（度，[0, 360)）
ang_diff(a, b)	两角之差归一化到 (-180, 180]，用于判断角楔包含关系
unit(a_deg)	单位向量 (cos a, sin a)

3.2 定位区域核心（68–260 行）

函数	算法与复杂度
wedge_halfplanes	见 2.1; 返回 2 个半平面, $O(1)$
halfplanes_satisfied	逐条检验 $ax + by \leq c$, 带相对容差; $O(m)$
intersect_lines	两直线求交, 用相对行列式判平行 ($ \det \leq 10^{-12} \max(\text{scale}, 1)$); $O(1)$
convex_hull	Andrew 单调链, 先按坐标去重 + 排序, 交叉积 $\leq$ 容差即弹出 (结果丢弃共线边点); $O(n \log n)$
polygon_vertices	算法主体: 枚举全部 $O(m^2)$ 条直线对的交点, 保留满足全部半平面的点, 再取凸包去重。 $m = 2n$ (n 个检测点), $n \leq 20$ 时最多 190 次求交, 满足在线计算需求
circular	求多个角区间的公共交集 (圆周上的区间交), 用于给出
arc_intersection	无界时的回收方向; $O(n \log n)$
is_bounded	回收锥判据: $\mathcal{R}$ 无界 $\iff$ 存在非零方向同时落在所有角楔内 $\iff$ 各角区间的交非空。与顶点枚举解耦, $O(n \log n)$
halfplanes_from_bearings	把 Stations+bearings 展开成半平面列表
region_from_bearings	总入口, 返回一个字典: vertices、empty、bounded、diameter、d_pair、cover_ok、cover_slack、mec (空集与无界情形提前返回, 不继续算直径)

为什么“顶点枚举”而不是传统半平面裁剪

凸集的顶点必为两条约束边界之交 (命题 1 的结构性结论), 因此直接枚举交点即可精确得到 $\mathcal{R}$, 不需要给定包围盒, 也避免了裁剪算法在无界情形下的数值困难。无界性由第 3 步的回收锥判断提前给出, 避免把无界区域误裁成有界区域。

3.3 直径与圆覆盖（265–347 行）

函数	说明
polygon_diameter	$O(m^2)$ 暴力枚举全部顶点对。 $m \leq 40$ 时完全够用, 且作为卡壳法的 校验基准 (两者最大差 0.0 m)
rotating_calipers_diameter	旋转卡壳, $O(m)$ (m 为凸包顶点数), 用于大规模场景
diameter_disk_slack	Thales 判据: 返回 $\max_V (V - A) \cdot (V - B) \leq 0$ 即以 AB 为直径的圆覆盖整个区域”。 $O(m)$

3.4 最小包围圆（349–403 行）

Welzl 随机增量算法, _circle_from2/_circle_from3/_inside 为其原语, 固定随机种子 seed=20260913 保证结果可复现。期望复杂度 $O(m)$ 。该圆半径即策略中使用的**位置不确定度** σ 。

3.5 问题二辅助（404–448 行）

`two_station_region`（两站定位区域的便捷封装）、`region_intersection_angle`（由三点的方位角算交会角 γ ）、`asymptotic_diameter`（实现式 $D \approx \sqrt{w_1^2 + w_2^2 + 2w_1w_2 \cos \gamma} / \sin \gamma$, $w_i = 2\epsilon d_i$ ）。

4 q2_solver.py 拆解

4.1 函数清单

函数	说明
<code>sector_points(S_1, θ_1)</code>	按题面约束生成可行源集合 $\mathcal{G}$ 的采样点：沿示向度方向、距离 $5 \sim r_{hi}$ 、方位角偏差 $\pm 1^\circ$ 、落在目标域内，返回 $(r, \text{phi}, (x, y))$ 列表
<code>worst_diameter($S_1, S_2, \mathcal{G}$)</code>	对 $\mathcal{G}$ 中每个源 G 计算两站定位区域直径，返回最大值（即最坏情形 D ）
<code>solve_second_point</code>	minimax 求解 ：外层对偏移距离 b 、内层对偏转角 φ 做网格搜索，返回 $b^*, \varphi^*, J^*, S_2^*$ ；带保证探测约束 $ S_2 - G \leq R_{\min}$
<code>monte_carlo_check</code>	4000 次随机抽样验证 J^* 是上界且无越界
<code>baseline_compare</code>	5 种第二检测点规则（A 本文/B 垂直偏移/C 沿示向度前进/D 随机/E 先知）的对照

4.2 求解流程

```

1 sector_points(S1, th1)           # 采样可行源集合 G (约 3000 点)
2 for b in 400..1500 step 50:      # 偏移距离
3     for phi in 5..60 step 1:     # 偏转角
4         S2 = S1 + b * unit(th1 + phi)
5         if not all(|S2 - G| <= R_min for G in G): continue # 保证探测约束
6         J = worst_diameter(S1, S2, G)
7     track argmin J
8 -> b*=1004.1 m, phi*=36.75 deg, J*=134.0 m

```

代码 3 问题二求解流程

4.3 基线策略的定义

为了让“本文规则更好”这一结论可检验，实现了四个基线：**B 垂直偏移 800 m**（把第二个点放在与示向度垂直的方向上）、**C 沿示向度前进 800 m**（自然的直觉做法）、**D 可行域内随机**、**E 先知策略**（直接站在真实源上方，给出性能下界）。代码在 `baseline_compare` 中对同一批随机算例逐一评估，结果写入 `data/q2_baselines.csv`。

5 robot_core.py 逐层拆解

这是最大的模块（1087 行），按职责分成 8 层。

5.1 第 1 层：两个客户端（55–140 行）

- HTTPClient: 真实走 HTTP。_post 负责 JSON 编码、request_id 管理、超时与错误码处理；四个方法 enter/exit/measure/clear 与协议一一对应。
- LocalClient: 接口完全相同的内存版客户端，直接调用 Arena。用于 60 组演练与参数调优（省掉 HTTP 往返，快一个数量级）。

为什么保留两个客户端

”能不能在真实协议下跑通”与”能不能大规模调参”是两个不同的需求。用同一套 Brain 代码 + 可替换的客户端，可以做到策略代码零修改地在本地高速模式与 HTTP 模式之间切换——正式测试时只换一行构造参数。

5.2 第 2 层：认证判据的原语（142–167 行）

_origin_inside_hull(points, tol): 判断原点是否严格位于点集凸包内部。实现为”凸包顶点按极角排序后，相邻顶点与原点构成的三角形面积同号”，等价于原点在每条边的内侧。这是问题四定理 2 的直接代码化。

5.3 第 3 层：参数与状态（169–250 行）

DEFAULT_PARAMS 共 30 个参数，分五组：

组别	参数	作用
信息获取	probe_spacing, probe_min_angle, max_channels_probe	移动多远补测一次、交会角多小才值得重测
定位门槛	locate_sigma	不确定度小于此值才进入清除阶段
末段清除	term_iters, term_cap, clear_try_radius, endgame_radius, endgame_step	归航迭代次数、单步上限、直接尝试清除的半径、图案清除的半径与步距
站位与规划	survey_mode, survey_spacing, ring_radius, search_cost_bias, clear_bonus, localise_bonus, probe_bonus	站位模式与间距、任务优先权偏置
预算与终止	max_attempts, hard_attempt_cap, progress_ratio, verify_grid, verify_r, outer_ring_gap	每频道清除尝试预算、认证网格与半径、域外补测环

状态字段包括：位置与时钟 pos、cur_channel、vtime；频道状态 status[c]（取值 unknown、seen、located、cleared、empty）；信息集合 obs[c]、nosig[c]、est[c]、last_seen[c]、

meas_cache; 以及 _task_count、_task_travel、_task_time（任务级统计，是发现故障 4 那类死循环的关键基础设施）。

5.4 第 4 层：站位生成（269–336 行）

函数	说明
_make_survey_stops	按 survey_spacing 生成格点（问题四用），三角形的交错行布置
_make_ring_stops	七点覆盖（问题三用）：中心 + 半径 ring_radius 的正六边形六顶点
_all_search_stops	按 survey_mode 合并，并可叠加域外补测环（outer_ring_gap > 0）
_verify_grid	认证用候选位置网格（间距 verify_grid，默认 60 m），仅保留在目标域内、且距边界留有余量的点

5.5 第 5 层：认证扫描（337–377 行）

certification_scan(channels) 是全代码中数学含量最高的一段：

```

1 R = verify_r - verify_grid * 0.7072    # 网格松弛，保证结论保守
2 for q in grid:
3     for c in channels:
4         near = [p - q for p in nosig[c] if |p - q| <= R]
5         if not near:                      flags[c] = False # 该处根本没测过
6         elif directional and not origin_inside_hull(near):
7             flags[c] = False # 存在方向可躲
8         else:                            flags.setdefault(c, True)
9 return (len(uncov) == 0), uncov, flags

```

代码 4 认证扫描的两种判据

关键设计：已经判定为 False 的频道直接跳过（第 361 行），因为一个未被认证的候选位置就足以否决整个频道，无需继续检验其余网格点。

5.6 第 6 层：估计与信息获取（413–592 行）

函数	说明
update_estimate	把 obs[c] 喂给 region_from_bearings；区域为空/无界时退化为 _best_pair（交会角最大的一对射线）的解，并按 $1/\sin\gamma$ 放大 σ
_best_pair	/ 退化情形的支撑原语
_ray_intersect	/
_intersection_angle	
_ray_limit	沿方位射线的可行长度上限（受目标域边界与 $R_{\max}$ 约束）
_worth_measuring	判断某频道在某位置是否值得测：已清除/已认证跳过；预测交会角小于 probe_min_angle 跳过（避免无效测量）

函数	说明
probe	在当前位置对全部”未解算”频道做一轮检测；按频道号排序以最小化切换时间
hear	移动到目标点测某频道；命中缓存时只付移动时间（reading 是地点函数，同点重测无新信息）——这一细节曾是一个严重 bug，见问题三技术报告 7.2

5.7 第 7 层：末段清除（593–733 行）

`clear_channel(c)` 是四阶段中的第三阶段，主循环 9 次迭代，每次按优先级：

1. $d \leq \text{clear_try_radius}(42 \text{ m}) \rightarrow$ 直接 `_try_clear`（未命中仅 3 s）；
2. $d \leq \text{endgame_radius}(90 \text{ m}) \rightarrow$ `_local_clear_sweep`（六边形图案，半径 0.55σ 截断在 $[12, 45] \text{ m}$ ）；
3. 若不在”上次听到信号的位置” $\rightarrow$ `hear` 一次；`near` 直接清、`no_signal` 则 `_retreat`（回退到上次听到信号处）；
4. 沿最后一条实测方位前进 $\min(0.7d, \text{term_cap})$ ，前进后：`near` $\rightarrow$ 清、`direction` $\rightarrow$ 更新估计并把步长放大 1.3 倍、`no_signal` $\rightarrow$ 步长 $\times 0.45$ 二分 + `_bracket_clear`（线段分点清除）；
5. 若目标点与当前点距离小于 5 m（已到边界被裁剪） $\rightarrow$ 随机方向走 8 m 打破僵局。

两个非平凡的设计

（1）二分收敛：每次过冲后步长按几何级数缩小，必然最终落入 20 m 清除半径内；**（2）放大 1.3 倍：**收到有效方位说明方向正确，主动加大步长以加速——这是一个”成功则加速、失败则减半”的自适应步长，实测比固定步长快约 30%。

5.8 第 8 层：任务规划与主循环（765–1145 行）

函数	说明
<code>_build_tasks</code>	生成候选任务列表：清除（已定位未清除）、定位（只有一条方位且 <code>vantage_tries</code> 未超限）、站位（覆盖站位未走完）
<code>next_search_stop</code>	无任务时决定”补测去哪”：贪心选 $\arg \max \frac{\text{新增认证覆盖}}{\text{行程} + \text{search_cost_bias}}$ ；问题四额外要求新点带来的观测方向与已有邻点夹角 $> 55^\circ$
<code>_plan_tour</code>	最近邻构造 + 2-opt 改进（最多 12 轮）。最近邻的距离度量中减去优先权偏置 <code>bonus</code> ，使清除任务在不远时被优先访问
<code>_run_planned</code>	主循环： <code>commit_estimates</code> $\rightarrow$ <code>all_done</code> 检查 $\rightarrow$ 构建/复用任务序列 $\rightarrow$ 执行首个任务 $\rightarrow$ 每移动 650 m 做一次机会式补测
<code>_run_greedy</code>	对照策略（纯贪心、无认证判据），供 § 实验对照使用

函数	说明
<code>run</code>	入口：原点普查 20 个频道 → 进入主循环；返回统计
<code>stats</code>	汇总：完成率、平均时间、任务级计数/行程/耗时（调试故障 4 的关键）

6 一次完整动作的调用链

以” 归航过程中前进 300 m 并收到有效方位” 为例，跟踪一次调用的完整路径：

```

1 Brain._run_planned()           # 主循环，选中 CLEAR 任务
2   -> Brain.clear_channel(c)     # 末段归航（第 3 阶段）
3     -> Brain.hear(c, self.pos)  # 确认当前点能听到
4       -> Brain.move_measure(x,y,c) # 记账 + 发请求
5         -> self.cl.measure(rid, x, y, c)
6           -> HTTPClient._post("/measure", {...}, rid)
7             -> sim._Handler.do_POST() # 协议校验
8               -> Arena.measure(...) # 物理解算
9                 <- {"measure_result": "direction", "svd_deg": 63.42, ...}
10                <- 更新 vtime / pos / cur_channel / 任务统计
11                -> Brain._handle_measure(c, pos, r) # 更新 obs/status/last_seen
12      -> 收到 no_signal? -> _retreat / _bracket_clear
13      -> 收到 direction? -> update_estimate -> region_from_bearings -> MEC -> est[c]
14      -> 前进 300 m: move_measure(...)
15        -> ... 同上 ...
16      -> 若 d_est <= 42 m -> _try_clear -> move_clear -> clear result

```

代码 5 调用链（自顶向下）

7 复杂度与性能

环节	复杂度	单次耗时（实测）	说明
定位区域求解	$O(m^2)$, $m = 2n \leq 40$	< 0.2 ms	每次估计更新调用一次
最小包围圆	$O(m)$ 期望	< 0.1 ms	Welzl
tour 规划	$O(k^2) \times 12$, $k \leq 40$	< 1 ms	每次任务变化后重规划
认证扫描	$O(grid \cdot C \cdot Z)$	2 ~ 15 ms	仅在无任务时调用
一局（内存模式）	—	0.3~0.6 s	60 组演练使用
一局（HTTP 模式）	—	1.3~2.3 s	约 1500 次请求

全部单局程序运行时间都远低于 20 min 上限（最坏 2.3 s，余量三个数量级）。

8 工程约束与规范

1. 可执行源码纯 ASCII，中文只在注释与外部数据文件。所有图内中文标签集中在一个外部文件里（zh_labels.json，100 个键），由 T(key) 读取，因此源码不会乱码，改标签也不必动代码。

-
2. **固定随机种子。**全部随机过程（算例生成、Welzl、Monte-Carlo、打散方向）都接受显式 `seed`，保证结果可复现。
 3. **实验脚本与策略代码分离。**`make_data.py` / `run_http_tests.py` 只负责“跑 + 落盘”，策略逻辑全部在 `robot_core.py` 内，便于同一策略在不同实验配置下复用。
 4. **任务级统计内建。**`stats()` 输出每类任务的次数/行程/耗时——这是发现“5957 次 LOCALISE 死循环”这类问题的唯一手段。
 5. **失败可诊断。**归航循环支持 `debug_channel` 开关，打开后逐迭代打印位置、估计距离、测量结果与步长比例。